

The background is a dark blue, out-of-focus image of network equipment. It shows several blue Ethernet cables plugged into ports, with some yellow and red indicator lights visible. The overall aesthetic is technical and modern.

NETWORKS AND SYSTEMS MANAGEMENT

Alexandre Cunha
up202400086

MOTIVATION

- As part of our LGP (Laboratório de Gestão de Projetos) class, we're simulating a startup that delivers a mobile-first solution for journalists to **share video reports on the go**.
- The app supports **mobile uploads** (recorded or ingested content), backed by a service architecture that includes:
 - A backend application
 - The tusd resumable upload server
 - All traffic managed by a nginx reverse proxy

MOTIVATION CONT.

- To ensure a **production-ready and scalable environment**, we identified key needs:
 - **Automated deployment** of new backend versions with minimal human intervention
 - **Dynamic nginx configuration updates** without downtime
 - **Robust monitoring** for observability and service health tracking
 - This setup not only serves our current simulated startup needs but also lays the groundwork for **container orchestration with Kubernetes**
-

PROBLEMS

Human Error: Manual deployment and configuration increases the risk of mistakes, such as incorrect environment settings or misconfigured services.

Deployment Inconsistency: Without automation, it's hard to guarantee that each deployment is identical, leading to unpredictable bugs and behavior.

Downtime During Updates: Manual changes to services like nginx can cause temporary service outages or downtime during reloads.

Compatibility Issues: Integrating new services (e.g., tUSD) without structured workflows may introduce version conflicts or misalignments.

MORE PROBLEMS

Lack of Scalability: Without automated scaling, the system cannot respond dynamically to changes in demand, leading to performance bottlenecks or resource waste.

Inefficient Rollbacks: In the case of a failed deployment, manually rolling back is slow and error-prone.

Poor Observability: Without monitoring integration, diagnosing issues requires digging through logs manually, which slows down incident response.

Onboarding Complexity: New team members face a steep learning curve when deploying or debugging the system manually.

SOLUTIONS

CI/CD Pipeline

- Automate backend deployments using continuous integration and delivery tools (e.g., GitHub Actions, GitLab CI, or Jenkins) to ensure consistency and reduce manual intervention.

Rollback Mechanisms

- Incorporate version tagging and automated rollback steps in the CI/CD process to quickly revert faulty deployments.

Dynamic nginx Configuration Reloads

- Use automation tools or hot-reloading strategies (like `nginx -s reload` or configuration templates) to update routes without downtime.

Infrastructure as Code (IaC)

- Define service configurations (e.g., using Docker Compose) to make deployments reproducible and version-controlled.

SOLUTIONS CONT.

Health Monitoring Stack

- Integrate a monitoring stack (e.g., Prometheus + Grafana + cAdvisor) to track system metrics, visualize performance, and alert on anomalies.

Containerization

- Containerize all services to ensure environment consistency and simplify dependency management across environments.

Auto-Scaling Readiness

- Design the architecture to be compatible with Kubernetes, enabling horizontal or vertical scaling in future iterations.

Service Discovery & Health Checks

- Implement service discovery and health checks (e.g., via nginx, load balancers, or K8s probes) to route traffic only to healthy instances.

KEY BENEFITS

- **Increased Reliability**
 - Automated deployments and dynamic configuration reduce downtime and eliminate human errors during updates.
- **Scalability by Design**
 - The architecture supports integration with Kubernetes, allowing for automatic scaling based on load.
- **Improved Observability**
 - Monitoring dashboards and health checks provide real-time visibility into service status and performance metrics.
- **Reproducible Environments**
 - Containerization and Infrastructure as Code make it easy to reproduce production setups locally or in staging.

TECH STACK APPLICATION SERVICES

Rust App – Main backend logic, handling uploads and processing.

tusd – Resumable upload server implementing the TUS protocol with webhook hooks into the Rust backend.

Workers – ML Models tailored to the business need

- Caption
- Quality
- Transcription

TECH STACK

NETWORK & PROXY

Nginx (OpenResty) – Acts as a reverse proxy to route external traffic to internal services, with future support for SSL and stub_status for metrics.

TECH STACK MONITORING

Prometheus – Collects system and application metrics from exporters.

Grafana – Dashboards and alerting based on Prometheus metrics.

cAdvisor – Provides detailed container-level resource usage metrics.

nginx-exporter – Exposes Nginx metrics (e.g., requests per second, connections) to Prometheus.

rust-prometheus – Export app metrics to Prometheus.

TECH STACK INFRASTRUCTURE

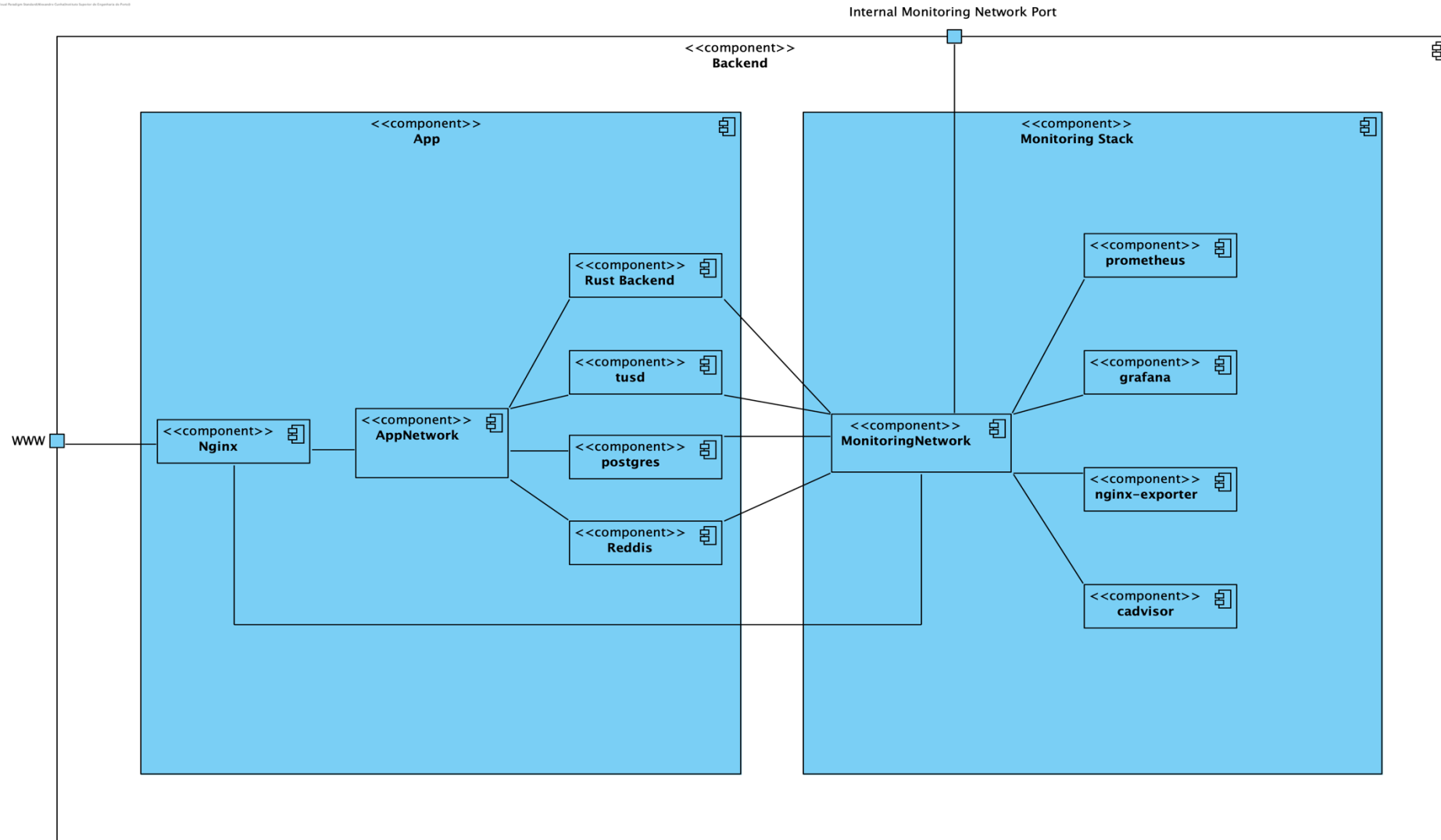
Docker & Docker Compose – Core
toolchain for service definition and
orchestration

Enabling:

- Multi-service architecture
- Inter-service communication via custom networks (tusc-network, mon-net)
- Mounting local directories for live development

ARCHITECTURAL OVERVIEW

Small text at the top left of the diagram area.



APP COMPOSE

The **base docker-compose file** defines and deploys the **main application stack**:

- **Rust App** – Core backend service for upload handling and business logic.
- **tusd** – Handles resumable file uploads; forwards post-finish hooks to the Rust backend.
- **Nginx (OpenResty)** – Acts as reverse proxy; routes requests to both app and tusd services.
- **workers** – do multiple analyses to the uploaded videos
- **Redis** – Supports caching or transient data if needed.
- **Postgres** – Database for persistence

MONITORING STACK COMPOSE

A separate **docker-compose** file is used to deploy the **monitoring stack**, which includes:

- **Prometheus** – Collects metrics from all instrumented containers.
- **Grafana** – Provides real-time dashboards for visualization and alerting.
- **cAdvisor** – Exposes resource usage metrics (CPU, memory, disk I/O) per container.
- **nginx-exporter** – Converts Nginx `stub_status` into Prometheus-friendly metrics.

Services

NGINX Status

UP

Backend App

ON

TUSD

ON

Redis

ON

Postgres

ON

Caption Worker

ON

Quality Worker

ON

Transcription Worker

ON

System

Running containers

14

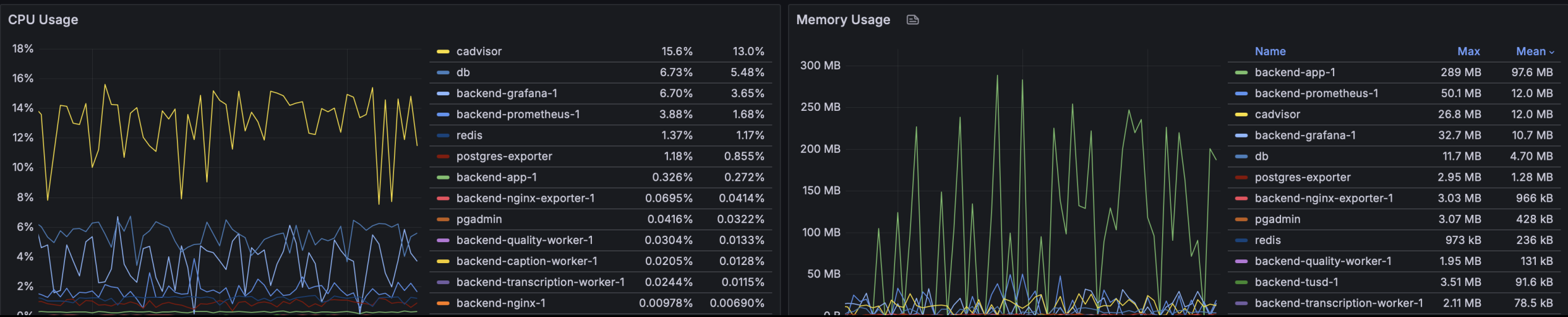
Total Memory Usage

1.53 GiB

Total CPU Usage

21.9%

Hardware Usage



NGINX

Active Connections



Total requests

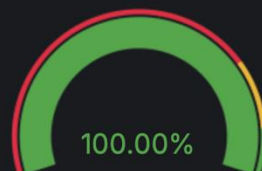


Postgres

Connections used



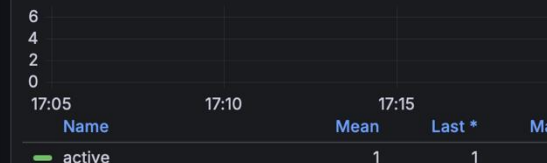
Commit Ratio



Deadlocks by database



Connections by state (stacked)



Other

Container Restarts



Services

NGINX Status
DOWN

Backend App
ON

TUSD
ON

Redis
ON

Postgres
2.00 mins

Caption Worker
ON

Quality Worker
ON

Transcription Worker
14.9 s

System

Running containers

14

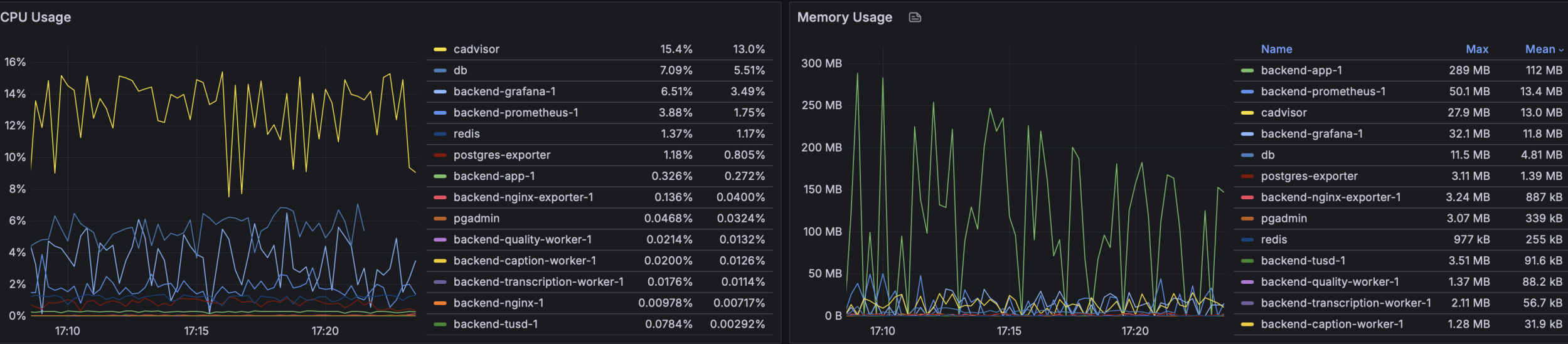
Total Memory Usage

1.29 GiB

Total CPU Usage

12.3%

Hardware Usage



Dashboard if services goes down

Running containers

Total Memory Usage

Total CPU Usage

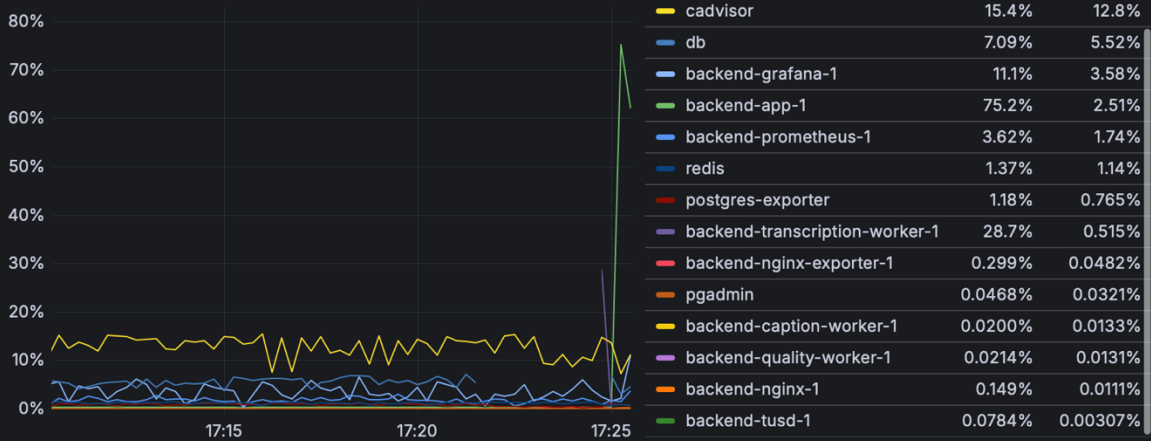
14

2.80 GiB

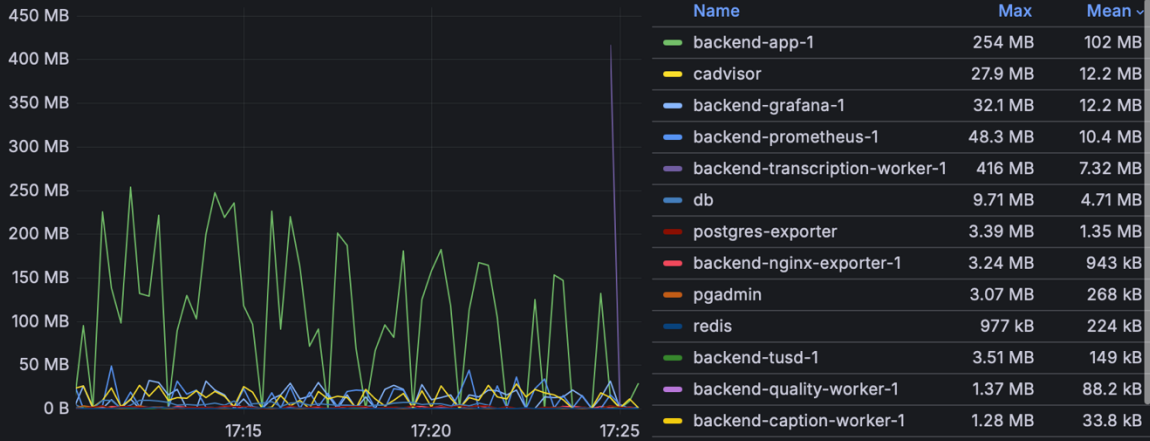
55.2%

Hardware Usage

CPU Usage



Memory Usage

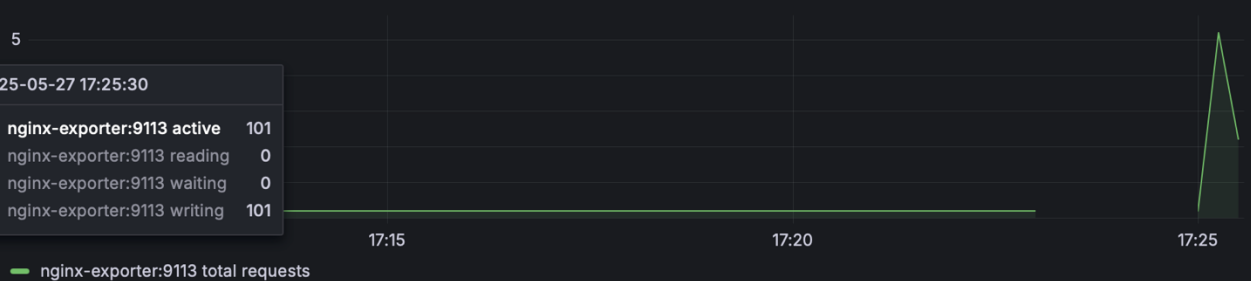


NGINX

Active Connections



Total requests



Postgres

Connections used

Commit Ratio

Deadlocks by database

Connections by state (stacked)

Dashboard if under load

STRESS TEST

LOAD TESTING

SUMMARY (K6)

- Duration: **2 minutes 30 seconds**
- Max Virtual Users (VUs): **100**
- Objective: Test system under extreme concurrency
- Test simulates: Account creation; Multiple logins; Video uploads

TOTAL RESULTS

HTTP

```
http_req_duration.....: avg=17.5s  min=23.13ms med=8.78s max=1m0s  p(90)=1m0s  p(95)=1m0s
  { expected_response:true }.....: avg=7.02s  min=23.13ms med=7.39s max=11.38s p(90)=10.8s p(95)=10.94s
http_req_failed.....: 29.36% 185 out of 630
http_reqs.....: 630 4.199922/s
```

EXECUTION

```
dropped_iterations.....: 378 2.519953/s
iteration_duration.....: avg=59.79s min=30.34s  med=1m0s  max=1m46s  p(90)=1m33s p(95)=1m39s
iterations.....: 185 1.233311/s
vus.....: 38 min=5 max=100
vus_max.....: 100 min=100 max=100
```

NETWORK

```
data_received.....: 313 kB 2.1 kB/s
data_sent.....: 2.3 MB 15 kB/s
```

running (2m30.0s), 000/100 VUs, 185 complete and 38 interrupted iterations
user flow ✓ [=====] 038/100 VUs 2m0s 5.00 iters/s

STRESS TEST

HTTP REQUEST

METRICS

HTTP Performance:

- Total Requests: **630**
 - Success Rate: **70.64%**
 - Failures: **29.36%** (185 failed)
 - Request Duration:
 - Average: **17.5s**
 - Median: **8.78s**
 - Max: **1 minute**
 - 95th Percentile: **1 minute**
 - Expected responses only:
 - Average: **7.02s**
 - Max: **11.38s**
-

STRESS TEST EXECUTION DETAILS

What is an Iteration?

An **iteration** is **one full execution** of the script by a virtual user (VU).

In this script, one iteration does:

- **Register** a new user
- **Login** 10 times
- **Upload** 5 files in PATCH chunks (2MB each)

Execution & Iteration Stats:

- Completed Iterations: **185**
- Dropped Iterations: **378**
- Iteration Duration:
 - Average: **59.79s**
 - Median: **1 minute**
 - 95th Percentile: **1m 39s**
- Virtual Users:
 - Active: **Up to 100**
 - Stable at 38 during peak

So out of the **total 563 iterations attempted**, only **185 ($\approx 32.8\%$) completed successfully** under load.

STRESS TEST

KEY OBSERVATIONS

System Behavior Under Load:

- App remained **resilient** under heavy load
 - Faced **performance bottlenecks** with DB driver:
 - Crashed under sustained peak
 - Required **manual restart**
 - Bottlenecks seen especially with **concurrent logins and video uploads**
-

STRESS TEST CONCLUSION

Final Thoughts & Realistic Expectations:

- This test simulated a **worst-case load**
 - **Expected real-world peak**: 20–30 concurrent users
 - App functioned for ~**1 minute** under full 100-user pressure
 - **Acceptable performance** for anticipated usage
 - Next steps:
 - Optimize DB interactions
 - Add **graceful failure handling**
 - Implement **automatic recovery**
-

END

Alexandre Cunha, up202400086

Q&A?
